

# ToolMinimize: Auditing and Rewriting LLM Agent Tool Calls to Minimize Privacy Exposure

Wenbiao Li

Case Western Reserve University  
Cleveland, OH, USA  
wxl387@case.edu

Yuqiao Xu

Case Western Reserve University  
Cleveland, OH, USA  
yxx914@case.edu

**Abstract**—LLM agents routinely include privacy-sensitive data (PSD) in tool call arguments beyond what the invoked tools require, crossing trust boundaries to third-party services on every invocation. A controlled measurement on three production LLMs (GPT-4o, Claude 3.5 Sonnet, Llama-3.3-70B) shows that 81–88% of tool calls include unnecessary PSD under default prompts; explicit privacy instructions still leave 36–76% over-sharing. Existing defenses gate calls (allow/block) or label flows (information-flow control) but cannot *rewrite* argument values, and PII detection tools miss implicit PSD like “Memorial Sloan Kettering” (a hospital name that implies a diagnosis). We present TOOLMINIMIZE, a middleware that intercepts tool calls and rewrites their arguments to the minimum data necessary for tool functionality, combining schema-aware necessity analysis with four operations: removal, generalization, substitution, and truncation. Live validation on 307 tool calls across the three LLMs above reduces privacy cost by 81.2–92.0% at 100% argument-level task validity (TOST equivalence  $p < 0.001$  at  $\Delta = 1.0$ ); on 25 unannotated Model Context Protocol (MCP) schemas, by 79.0% with no minimum\_necessary metadata. An optional LLM content-necessity layer strips task-irrelevant PSD from otherwise-necessary free-text fields, raising live-LLM reduction to 85.1–95.6% and author-schema reduction from 71.1% to 90.9%. Median latency is 1.77 ms.

**Index Terms**—privacy, LLM agents, tool use, data minimization, middleware, MCP

## I. INTRODUCTION

Consider a user who asks their LLM-powered assistant: “What will the weather be like near my oncologist at Memorial Sloan Kettering next Thursday?” The agent calls a weather API with location: “Memorial Sloan Kettering Cancer Center, 1275 York Avenue, New York, NY 10065”. The weather service needs only a city-level

location and a date; instead it receives the name of a cancer treatment center, revealing the user’s likely health condition to a third-party operator. This pattern is pervasive. An agent scheduling a meeting creates a calendar event titled “Chemotherapy Session – Dr. Patel, Oncology,” leaking a diagnosis to a calendar service. An agent helping locate a support group searches for “Alcoholics Anonymous meeting near 742 Evergreen Terrace, Springfield,” exposing both addiction status and home address. In each case, the tool functions correctly with far less data. The agent over-shares because it optimizes for task completion, not privacy.

We call this *tool call over-sharing*: the systematic inclusion of privacy-sensitive data (PSD) in tool call arguments beyond what tools require. The phenomenon is empirical, not theoretical. Across 20 everyday tasks on three production LLMs (GPT-4o, Claude 3.5 Sonnet, Llama-3.3-70B), 81–88% of tool calls contain unnecessary PSD under default system prompts (Section II); explicit privacy instructions cut this only to 36–76%, and Llama barely responds to them at all (81%→76%). Over-sharing crosses trust boundaries to third-party services with every invocation, is invisible to users, and is uncontrolled by existing frameworks. Every agent with tool-use capability [1], [2] is affected, and exposure scales linearly with the number of tool calls per task.

a) *Why existing defenses are not enough.*: **Allow/block gating**, whether call-level (AudAgent [3], PrivacyChecker [4]) or argument-level (Progent [5], which uses JSON Schema but still emits allow/block with fallback actions), cannot rewrite argument values: a weather query with “Memorial Sloan Kettering” either proceeds and leaks the facility or is refused, losing the task. **Information flow control** (Fides [6], RTBAS [7], CaMeL [8]) tracks confidentiality–integrity labels at flow granularity with capability-based allow/deny; it does not perform graduated rewriting of individual fields, and its label lattices are not designed for PSD sensitivity stratification. **PII detection tools** (Presidio, AWS Comprehend) are token-level pattern matchers that miss *implicit* PSD (“Memorial Sloan Kettering” is not a PII pattern but implies cancer) and are schema-unaware. *No existing system performs schema-aware, argument-level data minimization by rewriting tool*

Accepted for publication in the Proceedings of the 23rd Annual International Conference on Privacy, Security and Trust (PST 2026), University of Ottawa, Ottawa, Canada, August 26–28, 2026. This is the authors’ accepted version; the final published version will appear in IEEE Xplore (DOI to be added upon publication).

© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

call fields while preserving task validity, and none define a quantitative privacy cost metric for tool call exposure.

*b) Contributions.:* We present TOOLMINIMIZE, a middleware that intercepts LLM agent tool calls before execution, classifies PSD in their arguments, computes a quantitative privacy cost, and rewrites arguments to the minimum necessary data. It sits between the agent and the tool without changes to the LLM, framework, or tools. Over-sharing is an inference-time phenomenon: the helpfulness objective pushes the LLM to include all context in tool arguments regardless of safety training, and our motivating study confirms that privacy prompts still leave 36–76% of calls leaking PSD. Intervention therefore must act on the LLM’s *output*.

This paper contributes:

- 1) A practical taxonomy of 10 PSD categories grounded in GDPR Art. 9 classifications, and a quantitative privacy cost metric  $PC(t, args)$  combining data sensitivity, tool trust, and argument necessity. The metric is designed for *ordinal ranking* of methods: Kendall’s  $\tau = 1.0$  across 110 weight perturbations, so our main ranking result is invariant to parameterization.
- 2) TOOLMINIMIZE, to our knowledge the first system that applies *graduated rewriting* (removal, generalization, substitution, truncation) at argument granularity while preserving task validity. Progent [5] is also schema-aware over arguments but enforces via allow/block with fallback actions; TOOLMINIMIZE mutates field values instead. The pipeline combines pattern+semantic PSD classification, JSON Schema necessity analysis, and the four rewriting operations.
- 3) A controlled motivating study on three production LLMs (120 API calls) that establishes the empirical basis of tool call over-sharing, complementing concurrent benchmarks [9], [10], [11].
- 4) A comprehensive evaluation across three agent frameworks (AutoGen, MCP, LangChain), four LLM profiles, 90 author scenarios, 50 external scenarios, and 25 real-world MCP schemas. Live validation on 307 tool calls across GPT-4o, Claude Sonnet, and Llama-3.3-70B achieves 81.2–92.0% privacy cost reduction at 100% task validity (TOST  $p < 0.001$  at  $\Delta = 1.0$ ); on unannotated MCP schemas, 79.0%; on Stage C author scenarios, 73.6% ( $7.3\times$  lower residual PCS than PII Detection).
- 5) AgentPrivBench, an open-source benchmark with ground-truth PSD annotations and standardized privacy metrics.

## II. MOTIVATING STUDY

Before designing a mitigation, we ask: *do production LLMs actually over-share PSD in tool calls, and can safety training or privacy instructions prevent it?* Prior work motivates agent privacy risks from theoretical threat models [12], [13] or benchmark-only evaluations. Concurrent work corroborates the concern: AgentDAM [9] finds web-navigation agents are prone to unnecessary sensitive data use, TOP-Bench [10] reports a 90% average risk leakage rate via tool orchestration,

TABLE I  
MOTIVATING STUDY: PRODUCTION LLM OVER-SHARING RATES ON 20 REALISTIC AGENT TASKS. *Over-share%* = FRACTION OF CALLS WITH  $\geq 1$  UNNECESSARY PSD ITEM; *Leakage%* = FRACTION OF POSSIBLE UNNECESSARY ITEMS ACTUALLY LEAKED.

| Model             | Condition | Over-share% | Avg Unnec. | Leakage% |
|-------------------|-----------|-------------|------------|----------|
| GPT-4o            | Default   | 81.5%       | 1.00       | 40.9%    |
|                   | Privacy   | 36.0%       | 0.40       | 15.9%    |
| Claude 3.5 Sonnet | Default   | 87.5%       | 0.94       | 34.9%    |
|                   | Privacy   | 50.0%       | 0.56       | 20.4%    |
| Llama-3.3-70B     | Default   | 81.0%       | 1.00       | 38.9%    |
|                   | Privacy   | 76.2%       | 0.86       | 33.3%    |

and AgentLeak [11] measures 69% system exposure in multi-agent configurations. We complement these with the first controlled measurement of *tool call argument-level* over-sharing by production LLMs.

*a) Methodology.:* We constructed 20 everyday agent requests that naturally contain personal information, the kind of tasks users routinely delegate (e.g., “check the weather near my doctor’s office,” “email my manager asking for a day off”). Each prompt specifies 1–2 tools from a standard set (weather, calendar, search, email, maps) with ground-truth annotations of *necessary* vs. *unnecessary* PSD. We queried three production LLMs (GPT-4o, Claude 3.5 Sonnet, Llama-3.3-70B) via function-calling APIs at temperature 0, under two conditions: *Default* (standard helpful system prompt) and *Privacy* (explicit instructions to minimize PSD with concrete examples). This yields  $3 \times 20 \times 2 = 120$  API calls, each annotated for unnecessary PSD.

*b) Over-sharing is pervasive.:* Under the default prompt, **81–88% of tool calls contain at least one unnecessary PSD item** (Table I). All three models over-share at similar rates despite differing safety training: GPT-4o (81.5%), Claude (87.5%), Llama (81.0%). In 10 of 20 scenarios, all three models unanimously over-share; in 15 of 20, at least two do.

Representative examples: a weather query about a cardiology appointment produces `location`=“9500 Euclid Avenue, Cleveland OH” with `context`=“cardiology appointment at 2pm”; the API needs only “Cleveland, OH.” All three models title a couples-counseling event “Couples Counseling with Dr. Patricia Hoffman” instead of a generic “Appointment.” When asked to email a manager about a day off, all three include “severe anxiety and panic attacks since the layoffs.” One model sends a user’s SSN and income to an accountant.

*c) Privacy instructions help some models, but not enough.:* Explicit privacy prompts reduce over-sharing for GPT-4o (81.5%→36.0%) and Claude (87.5%→50.0%), but have minimal effect on Llama (81.0%→76.2%). Even in the best case (GPT-4o with privacy prompt), over one-third of tool calls still contain unnecessary PSD. Five scenarios resist all prompting: addiction searches, custody queries, STI clinic lookups, home-to-rehab directions, bankruptcy searches.

*d) Safety training is necessary but insufficient.:* RLHF and safety fine-tuning help but cannot remove the structural

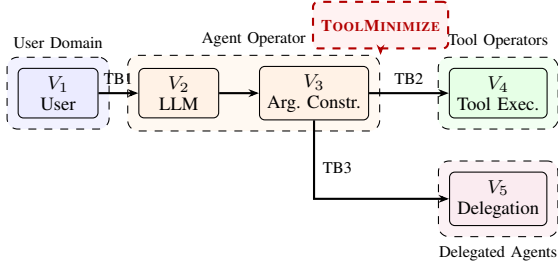


Fig. 1. LLM agent pipeline. Shaded regions denote trust domains. TOOLMINIMIZE intercepts tool calls at the agent-tool boundary (dashed box), between argument construction ( $V_3$ ) and tool execution ( $V_4$ ).

incentive to include all available context. The helpfulness that makes agents effective is the same behavior that leaks PSD, which motivates middleware operating on the agent’s *output* rather than relying on the model to self-censor.

### III. THREAT MODEL

#### A. System Model

An LLM agent processes a user request through five stages: user input ( $V_1$ ), LLM reasoning ( $V_2$ ), tool argument construction ( $V_3$ ), tool execution ( $V_4$ ), and cross-agent delegation ( $V_5$ ). Figure 1 illustrates the pipeline. Three trust boundaries partition it. The user-agent boundary ( $V_1 \rightarrow V_2$ ) is trusted: the user chooses to interact. The agent-tool boundary ( $V_3 \rightarrow V_4$ ) is semi-trusted: the agent operator selects the tool, but the tool operator is a distinct entity with its own data policies; a single request may trigger 5–20 tool calls [12]. The agent-agent boundary ( $V_2 \rightarrow V_5$ ) has variable trust: the user consented only to the primary agent’s practices.

TOOLMINIMIZE sits at the agent-tool boundary and intercepts every tool call after argument construction ( $V_3$ ) but before execution ( $V_4$ ); it requires no modifications to the LLM, framework, or tools. For highest accuracy, tool schemas should carry `minimum_necessary` annotations; without them, the analyzer falls back to the standard `required/optional` distinction.

#### B. Adversary Model

We consider three adversary classes. *Semi-honest tool operators* follow the protocol but log and analyze received argument data for profiling, resale, or surveillance (the honest-but-curious model [12], [13], [14]). *Network observers* passively observe tool call payloads on the network path, including via corporate proxies or API gateways that terminate TLS [15]. *Multi-tool aggregators* correlate argument data across tool calls from the same session to reconstruct user profiles [16].

**Out of scope.** Malicious LLM providers, adversarial users, prompt injection attacks, and training data memorization are orthogonal threats addressed by complementary systems [8], [6], [17].

#### C. Privacy Threats

We identify five privacy threats at agent-tool and agent-agent boundaries:

**C-1: Tool call over-sharing.** The LLM includes PSD beyond what the tool requires; this is the most pervasive threat (Section II). Free-text fields (queries, descriptions, notes) are the primary vectors.

**C-2: Tool-level data exfiltration.** A semi-honest operator extracts PSD from received arguments for profiling, resale, or surveillance. The tool functions correctly; the operator simply retains data it legitimately receives.

**C-3: Cross-tool aggregation.** A multi-tool aggregator correlates PSD fragments across calls to reconstruct user profiles that no single call exposes [16].

**E-1: Delegation-chain privacy degradation.** Delegation payloads typically include the full conversational context even when the subtask requires only a narrow slice, exposing PSD to agents under weaker privacy policies [18], [16].

**E-2: Context over-forwarding.** Mirrors C-1 at the agent-agent boundary [19]: the receiving agent may itself invoke tools, creating transitive exposure chains across multiple trust boundaries.

#### D. Scope and Assumptions

TOOLMINIMIZE targets **C-1 (tool call over-sharing)** as its primary threat. C-1 is the most pervasive privacy threat: it affects every tool call, crosses trust boundaries with each invocation, and is invisible to users. By reducing PSD in arguments, TOOLMINIMIZE also mitigates C-2 (reducing data available for exfiltration) and limits fragments available for C-3 (cross-tool aggregation). We evaluate effectiveness against delegation threats (E-1, E-2) in Section V.

We assume: (1) tools expose JSON Schema input schemas [14] (tools without schemas receive conservative defaults); (2) arguments are JSON key-value pairs; (3) the framework exposes a hook between argument construction and execution; and (4) tool success is observable from the response. Training data privacy [17], conversational disclosure, and prompt injection [8] are orthogonal threats.

a) *What TOOLMINIMIZE guarantees.* Every PSD item detected and marked unnecessary is removed or generalized before transmission. Residual exposure from classifier misses is bounded by  $\sum_c (1-r_c) n_c^{\text{unnec}} S(c) E(f)$ , with per-category recall  $r_c$  from 0.43 (DI) to 0.70 (L) (Section V-D); taking the worst case  $n_c^{\text{unnec}} = n_c$  (every missed item assumed in an unnecessary field) yields 4.09 PCS/scenario on AgentPrivBench Stage C, vs. observed 0.59 (6.9 $\times$  headroom). Adversarial encodings, out-of-taxonomy categories, necessary-field PSD retained by handlers, and rewriter failures on required fields (4/20 maps scenarios) fall outside this bound.

## IV. SYSTEM DESIGN

#### A. Pipeline Overview

TOOLMINIMIZE intercepts tool calls after LLM generation and before execution. When an agent emits a call with name  $t$  and arguments  $args$ , the middleware passes it through four stages (Figure 2): **Classify** (identify PSD items and categorize

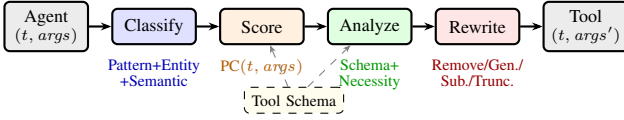


Fig. 2. TOOLMINIMIZE pipeline. Tool calls are intercepted after LLM generation and before execution.

TABLE II  
PSD TAXONOMY: 10 CATEGORIES WITH CODES, SENSITIVITY LEVELS, AND EXAMPLES.

| Code | Category           | $S$ | GDPR       | Examples                  |
|------|--------------------|-----|------------|---------------------------|
| DI   | Direct identifiers | 4   | Art. 4(1)  | SSN, email, phone, name   |
| H    | Health             | 4   | Art. 9(1)  | Conditions, medications   |
| F    | Financial          | 4   | Art. 4(1)  | Account numbers, income   |
| QI   | Quasi-identifiers  | 3   | Art. 4(1)  | Age, ZIP, occupation, DOB |
| L    | Location           | 3   | Art. 4(1)  | Street addresses, GPS     |
| R    | Relational         | 3   | Art. 9(1)  | Family, associations      |
| B    | Behavioral         | 2   | Art. 9(1)  | Habits, substance use     |
| Tp   | Temporal patterns  | 2   | Art. 4(1)  | Schedules, routines       |
| I    | Intent             | 2   | Recital 47 | Inferred goals, plans     |
| Im   | Implicit context   | 1   | Recital 26 | Contextual implications   |

them), **Score** (compute privacy cost  $PC(t, args)$ ), **Analyze** (determine field dispositions from the tool schema and necessity heuristics), and **Rewrite** (produce minimized arguments  $args'$ ). Scoring precedes analysis so every tool call is logged even when rewriting is disabled.

### B. PSD Taxonomy

We define a taxonomy of 10 PSD categories grounded in GDPR Article 9 and empirical analysis of our corpus (Table II). We use *PSD* rather than *PII* because tool arguments frequently contain data that is privacy-sensitive but not personally identifying in isolation: “Memorial Sloan Kettering” implies a health condition, and temporal patterns reveal routines. Sensitivity levels  $S \in \{1, 2, 3, 4\}$  are informed by GDPR regulatory tiers: Art. 9 special categories receive  $S=4$ , quasi-identifiers [20] receive  $S=3$ , and contextual data receive  $S \leq 2$ . These levels are a technical approximation of harm, not a compliance determination. The taxonomy is extensible; a sensitivity analysis confirming robustness to category exclusion appears in Section V-B.

### C. Privacy Cost Metric

We define the *Privacy Cost* of a tool call as

$$PC(t, args) = \sum_{j=1}^{|\mathcal{P}|} S(p_j) \cdot E(f) \cdot N(p_j, f) \quad (1)$$

where  $\mathcal{P}$  is the set of PSD items detected in  $args$ , and  $f$  is the server hosting  $t$ . **Sensitivity**  $S(p)$  maps each PSD item to its category level (Table II). **Exposure**  $E(f)$  captures the trust relationship on four levels: `FIRST_PARTY_LOCAL` (0.2, on-device), `FIRST_PARTY_REMOTE` (0.5, same provider), `THIRD_PARTY_DPA` (0.8, contracted),

`THIRD_PARTY_NO_DPA` (1.0, uncontracted), informed by the MCP trust hierarchy [14]. **Necessity**  $N(p, f)$  is binary: 0 if the containing field is in the tool’s `minimum_necessary` set (derived from schema annotations), 1 otherwise. Only unnecessary PSD drives the score. When schemas lack annotations, the system treats required fields as necessary and optional as unnecessary.

The metric is additive. Interaction effects (for example, a name plus a health condition is more revealing than either alone) are not modeled, but the median call has 2 categories so the effect is bounded in practice. The base metric satisfies non-negativity, monotonicity, and additivity. Our main acceptance result—that TOOLMINIMIZE dominates all utility-preserving baselines—holds for *any* monotone  $S$  and  $E$  mappings that preserve ordinal rankings: Kendall’s  $\tau = 1.0$  across 110 weight perturbations (Section VI-C).

a) *Worked example.*: A weather API call (`THIRD_PARTY_NO_DPA`,  $E=1.0$ ) with `location = “Memorial Sloan Kettering Cancer Center, 1275 York Ave, NY 10065.”` The classifier detects three PSD items at non-overlapping spans: facility name (H,  $S=4$ ), street address (L,  $S=3$ ), ZIP (L,  $S=3$ ). The schema specifies `location` as required with `minimum_necessary = [city-level]`, so  $N=1$  for all three:  $PC = 4 + 3 + 3 = 10.0$ . After generalization to “New York, NY,” only a city-level L detection remains ( $N=0$ ), yielding  $PC = 0.0$ .

### D. Classifier, Analyzer, and Rewriter

a) *Two-stage classifier.*: Stage 1 combines a *PatternClassifier* (40+ regexes covering DI, H, F, L, and additional categories) with an *EntityExtractor* (regex-based entity extraction mapped to PSD categories). Stage 2 (*SemanticClassifier*) catches PSD that patterns miss via sensitive facility lookup, government ID patterns, profession-based inference (e.g., “bankruptcy attorney”  $\rightarrow$  F), and cross-field combination detection. A base64-decoding preprocessor scans string arguments for encoded substrings; decoded spans are classified and re-mapped to their source ranges, defending against simple LLM-assisted encoding evasions (Section V-D). We use deterministic regex rather than statistical NER for zero external dependencies and sub-millisecond latency. An optional spaCy NER backend (Section V-D) raises DI recall from 0.435 to 0.739 at +26 ms, which we recommend for deployments requiring higher coverage.

b) *Analyzer.*: The analyzer decides each field’s disposition in two phases. Phase 1 (*SchemaAnalyzer*) parses each tool’s JSON Schema to extract required, optional, and minimum-necessary field sets; optional PSD-bearing fields receive REMOVE. Phase 2 (*SemanticNecessityAnalyzer*) examines required fields via tool-type-specific handlers: weather fields are generalized to city level, calendar titles are replaced with generic placeholders, search queries have identifiers stripped, maps queries have facility names stripped, and email fields are flagged for review. A generic handler covers 62.2% of scenarios with conservative rules: free-text fields with critical-sensitivity PSD are generalized; optional PSD

fields are removed. An optional `task_context` parameter passes user-intent phrases that selectively retain consented PSD (unused in our benchmark).

*c) Rewriter.*: Four operations correspond to analyzer dispositions: *Removal* (delete optional unnecessary fields), *Generalization* (coarsen values, e.g., “1275 York Ave, NY 10065”  $\rightarrow$  “New York, NY”), *Substitution* (insert synthetic but schema-valid values like `user@example.com`), and *Truncation* (round GPS, shorten ZIP+4 to ZIP-5). Three failure modes are supported: *fail-closed* (block the call and raise an error; default, per privacy-by-default [21]), *fail-open* (forward original, log incident; explicit opt-in), and *fail-prompt* (forward with user-confirmation flag). On initialization, the middleware flags schemas where *all* fields are `minimum_necessary` as suspicious, defending against semi-honest operators who might neutralize minimization via schema manipulation.

## E. Framework Integration

TOOLMINIMIZE integrates with three major agent frameworks through thin adapter layers sharing a common `ToolMinimizeMiddleware` core. `AutoGenToolMinimize` wraps registered tool functions via a decorator or agent-level patch; `MCPToolMinimize` intercepts `tools/call` JSON-RPC requests at the MCP client transport; `LangChainToolMinimize` wraps `BaseTool` instances in a proxy. All adapters share the same pipeline; our evaluation confirms identical privacy results across frameworks (Section V-D).

## V. EVALUATION

We evaluate TOOLMINIMIZE on a benchmark of 90 realistic agent scenarios addressing three research questions. **RQ1**: How pervasive is PSD over-sharing? **RQ2**: How does TOOLMINIMIZE compare with existing mitigations? **RQ3**: What is the contribution of each pipeline component?

### A. Experimental Setup

*a) Benchmark.*: *AgentPrivBench* contains 90 hand-curated multi-step scenarios covering healthcare (23), personal (19), finance (18), workplace (14), travel (12), and legal (4). Each scenario specifies a user request, tool calls, and ground-truth annotations of necessary vs. unnecessary PSD. Scenarios partition into *tool\_call* (Stage C, 60 scenarios, 73 tool calls), *cross\_agent* (Stage E, 15), and *reasoning* (Stage R, 15). Annotations follow a written rubric (GDPR Art. 9 tiers, per-tool minimum-necessary analysis). We addressed annotation-circularity concerns with two independent checks. (i) GPT-4o and Claude independently annotated 35 calls; both detect more PSD than our labels (5.3/7.6 vs. 3.4 items) at high correlation ( $\rho=0.63, 0.79$ ;  $p<0.001$ ), i.e., our labels are conservative rather than inflated. (ii) In Section V-D we evaluate on 50 PrivacyLens-style external scenarios with no shared annotations, where TOOLMINIMIZE achieves 89.1% cost reduction.

*b) LLM profiles.*: We evaluated under four simulated profiles modeled after GPT-4o, Claude 3.5, Llama-3.1-70B, and Qwen2.5-72B, each applying deterministic parameterized mutations to scenario-defined tool calls, varying optional-field inclusion, verbosity, and structure. This isolates TOOLMINIMIZE’s effect from LLM variance; we validate with live LLM inference on the newer GPT-4o, Claude Sonnet, and Llama-3.3-70B in Section V-D.

*c) Baselines.*: Nine methods: *No Mitigation*; three *Prompt* strategies (instruction  $p=0.20$ , few-shot  $p=0.35$ , system  $p=0.40$ , calibrated from [22], [23]); *PII Detection* (Presidio-style regex + named-entity recognition (NER)); *PrivacyChecker* [4] (contextual-integrity (CI) gating that blocks any call carrying critical PSD to third-party tools; since 97% of scenarios contain critical PSD,  $TCR = 3\%$  by design); *PrivacyChecker-relaxed* (blocks only on H + F, not DI;  $TCR = 37\%$ ); *AudAgent* [3] (re-implementation of its blocking mode); and TOOLMINIMIZE (rule-based backends, no LLM inference).

*d) Metrics.*: *PER*: fraction of calls with any PSD after rewriting. *PCS*: base  $PC(t, args)$  averaged per scenario. *DMS* (Data Minimization Score): mean per-scenario removal rate; bounded by the fraction of unnecessary PSD (theoretical max  $\approx 61.6\%$ ). *TCR*: fraction of scenarios with schema-valid, minimum-necessary-complete rewritten arguments—*argument-level* validity, not end-to-end task success (which we measure separately by execution in Section V-D and discuss in Section VI-B). Stochastic methods use  $n=5$  runs with seeds {42, 123, 456, 789, 2024} and report 95% CIs; pairwise comparisons use Wilcoxon signed-rank on  $n=60$  per-scenario PCS pairs. Experiments ran on Intel Xeon Gold 6248R, Python 3.11, no GPU.

### B. RQ1: Over-Sharing Is Pervasive

Across all four LLM profiles, baseline PER averages 0.997 with 5.88 PSD items per call ( $\approx 3.7$  unnecessary on average) and raw PCS 13.98. The near-identical exposure across profiles (range 13.62–14.15) supports the claim that over-sharing is structural. Healthcare has the highest raw PCS (18.18); personal has the highest density (7.71 items/call); even legal, the lowest-density domain (2.75 items/call), transmits unnecessary PSD in every invocation.

*a) Category distribution.*: 69.4% of PSD items are in critical-sensitivity categories (DI 33.0%, H 26.6%, F 9.8%). Health-related PSD appears across *all* domains: agents include medical context in travel, workplace, and financial queries. Even restricting the taxonomy to the six highest-sensitivity categories (excluding I, Im, Tp, B, 5.9% of items), PER remains above 0.99, confirming that baseline over-sharing is not an artifact of broad taxonomy definitions.

### C. RQ2: Mitigation Comparison

Table III compares the nine methods on 60 Stage C scenarios. Prompt-based methods yield modest reductions (best: 33.2%, PER still  $>0.96$ ). PII Detection reduces PCS to 4.29 (27.6% reduction) but misses non-entity PSD. PrivacyChecker

TABLE III

COMPARISON ON 60 STAGE C SCENARIOS ( $n=5$  RUNS FOR PROMPT METHODS). TOOLMINIMIZE ACHIEVES THE LOWEST PCS AT TCR = 100% ( $p < 0.001$  VS. UTILITY-PRESERVING BASELINES, WILCOXON). PROAGENT [5] IS BRACKETED BY PRIVACYCHECKER AND PRIVCHK-RELAXED (TABLE VI).  $\pm$  = 95% CI. **BOLD** = BEST AT TCR = 100%.

| Method              | PER $\downarrow$ | PCS $\downarrow$ | TCR $\uparrow$ | DMS $\uparrow$ | Red. $\uparrow$ |
|---------------------|------------------|------------------|----------------|----------------|-----------------|
| No Mitigation       | 1.000            | 11.68            | 1.00           | 0.0%           | 0.0%            |
| Prompt (instr.)     | 0.99 $\pm$ .01   | 9.27 $\pm$ .62   | 1.00           | 20.8%          | 17.2%           |
| Prompt (few-shot)   | 0.98 $\pm$ .01   | 7.29 $\pm$ .64   | 1.00           | 35.7%          | 29.8%           |
| Prompt (system)     | 0.97 $\pm$ .02   | 6.71 $\pm$ .61   | 1.00           | 40.4%          | 33.2%           |
| PII Detection       | 0.717            | 4.29             | 1.00           | 46.1%          | 27.6%           |
| PrivacyChecker      | 0.050            | 0.47             | 0.03           | 95.0%          | 73.2%           |
| PrivChk-relaxed     | 0.370            | 3.97             | 0.37           | 63.0%          | 63.9%           |
| AudAgent (detect)   | 1.000            | 11.68            | 1.00           | 0.0%           | 0.0%            |
| <b>ToolMinimize</b> | <b>0.567</b>     | <b>0.59</b>      | <b>1.00</b>    | <b>62.3%</b>   | <b>73.6%</b>    |

achieves PCS = 0.47 at catastrophic utility loss (TCR = 3%); relaxing to block only on H + F raises TCR to 37% but PCS to 3.97. AudAgent’s blocking mode (our re-implementation) acts post-hoc on detected items but does not rewrite argument fields.

TOOLMINIMIZE achieves PCS = 0.59, close to PrivacyChecker’s 0.47 but with 100% task completion versus PrivacyChecker’s 3%. The residual 0.59 corresponds to PSD that is genuinely necessary for the task (e.g., passenger names for flight booking). On residual PCS, TOOLMINIMIZE is  $7.3\times$  lower than PII Detection and achieves 16.2pp higher DMS (62.3% vs. 46.1%) because it targets the items that should be removed: PII Detection misses implicit PSD and over-retains detected items in required fields.

*a) Per-domain and tradeoff.*: TOOLMINIMIZE achieves near-zero PCS in four of five Stage C domains (finance 0.08, healthcare 0.43, personal 0.42, workplace 0.00); travel (2.00) is the exception due to named-entity identifiers that are minimum-necessary for booking. On Stage E (15 delegation scenarios), TOOLMINIMIZE achieves PCS = 0.0 at 100% TCR. TOOLMINIMIZE and PrivacyChecker form the Pareto frontier (PrivacyChecker has lower PCS but TCR = 3%); all other methods are dominated ( $p < 0.001$ , Wilcoxon). These baselines pursue partly different objectives—call-level gating (AudAgent), contextual-integrity policy enforcement (PrivacyChecker), and token redaction (PII Detection)—so the comparison isolates a single axis: argument-level minimization that preserves task validity. We do not claim general superiority over agent-security systems with different goals (e.g., information-flow integrity); TOOLMINIMIZE is complementary to them (Section VII).

#### D. RQ3: Ablation and Live Validation

*a) Classifier ablation.*: Table IV shows incremental contributions. Pattern matching alone detects 290 PSD items; entity extraction adds 58.6% more (to 460). The semantic classifier contributes only 4 additional detections. The semantic necessity analyzer is the larger lever: schema-only necessity

TABLE IV

ABLATION ON 60 STAGE C SCENARIOS. ENTITY EXTRACTION ADDS 58.6% MORE DETECTIONS; SEMANTIC NECESSITY ADDS 44 REMOVALS.

| Configuration        | Det. | Rem. | Rem. Rate | TCR  |
|----------------------|------|------|-----------|------|
| Pattern-only         | 290  | 156  | 0.538     | 1.00 |
| Pattern + Entity     | 460  | 274  | 0.596     | 1.00 |
| Schema-only analysis | 464  | 234  | 0.504     | 1.00 |
| Full ToolMinimize    | 464  | 278  | 0.599     | 1.00 |

achieves 0.504 removal rate (234) vs. 0.599 full pipeline (278), generalizing PSD within required fields.

*b) NER configuration.*: DI is the hardest category for regex classification (recall 0.435). Adding spaCy NER (en\_core\_web\_sm) raises DI recall to 0.739 at +26 ms median latency. We report regex-only in comparison tables (for reproducibility) and enable NER for metric validation.

*c) Real-world and external validation.*: On 25 MCP schemas from public servers (Shopify, GitHub, Discord, Twilio, Zoom, Jira, and 19 others) with 53 scenarios (none having minimum\_necessary annotations), TOOLMINIMIZE achieves **79.0% aggregate cost reduction**. On 50 PrivacyLens-style external scenarios not tailored to our handlers [22], TOOLMINIMIZE achieves **89.1% cost reduction** (PER 1.0  $\rightarrow$  0.16), with TCR 60% (external minimum-necessary definitions do not always align with schemas). On 20 benign scenarios, detection FPR is 20% (4/20) and rewrite FPR 10%.

*d) Production PII tool comparison and adversarial evasion.*: We ran Presidio, AWS Comprehend, and Google Cloud DLP on all 73 Stage C calls. TOOLMINIMIZE detects more items per call (regex 6.30, NER 8.56) than Presidio (4.70), Comprehend (3.30), or DLP (4.42), at 50–100 $\times$  lower latency; none of the three is schema-aware or rewrites arguments. We also tested 10 PSD encoding strategies (base64, homoglyphs, leetspeak, field splitting, ...); with a base64 decoding preprocessor, 9/10 trigger detection. Only field splitting fully evades; compositional detection is future work.

*e) Live LLM validation.*: We ran 90 scenarios through GPT-4o, Claude Sonnet, and Llama-3.3-70B, yielding 307 tool calls across 270 (scenario, model) pairs (Table V). Schema-only TOOLMINIMIZE achieves 81.2–92.0% cost reduction with 100% task completion across all three models; the content-aware extension raises this to 85.1–95.6% (mean 91.6%) by stripping task-irrelevant PSD from free-text fields the LLM populates. Mann-Whitney U finds no significant difference between real and simulated PCS ( $p=0.085$ ); TOST confirms equivalence at  $\Delta=1.0$  ( $p < 0.001$ ). At tighter bounds the equivalence claim weakens:  $\Delta=0.5$  still passes ( $p=0.046$ ), but  $\Delta=0.25$  does not ( $p=0.23$ ).

*f) Content-aware extension.*: The schema-only pipeline treats any field marked minimum\_necessary as fully necessary and does not remove PSD from within it, so send\_email and send\_message scenarios register PC = 0 and 0% reduction even when the body contains task-irrelevant disclosures (“recent job loss,” “unemployment benefits”). We add an optional



TABLE V

LIVE LLM VALIDATION ON THREE PRODUCTION LLMs (ALL 90 SCENARIOS, 270 (SCENARIO, MODEL) PAIRS, 307 TOOL CALLS). TOST EQUIVALENCE  $p < 0.001$  AT  $\Delta = 1.0$ . **RED.%** IS SCHEMA-ONLY TOOLMINIMIZE; **RED.% (CA)** ADDS THE CONTENT-AWARE EXTENSION. TCR IS 100% THROUGHOUT. BASELINE PCS GAP (SIMULATED 11.68 VS. REAL 4.10) REFLECTS SIMULATED PROFILES INCLUDING ALL OPTIONAL FIELDS; POST-MINIMIZATION PCS CONVERGES (0.46 VS. 0.59).

| Model            | PCS <sub>bet</sub> | PCS <sub>aft</sub> | Red.%        | Red.% (CA)   |
|------------------|--------------------|--------------------|--------------|--------------|
| GPT-4o           | 3.95               | 0.40               | 90.0%        | 94.1%        |
| Claude Sonnet    | 5.20               | 0.42               | 92.0%        | 95.6%        |
| Llama-3.3-70B    | 3.05               | 0.57               | 81.2%        | 85.1%        |
| <b>Average</b>   | 4.10               | 0.46               | <b>88.9%</b> | <b>91.6%</b> |
| Simulated (main) | 11.68              | 0.59               | 73.6%        | 90.9%        |

per-item necessity analyzer that asks an LLM (GPT-4o), for each detected PSD item, whether the specific value is essential to the user’s task; items judged unnecessary are stripped from the field while the surrounding text is preserved. On the same 90 scenarios (103 tool calls), content-aware TOOLMINIMIZE achieves **90.9%** mean cost reduction versus 71.1% schema-only: all 14 send\_email and 7 send\_message calls now reach 100% reduction, and 22 previously PC = 0 scenarios become nonzero. Maps scenarios trade some reduction (100%→80%) because the content analyzer correctly flags origin/destination addresses as task-essential. Adding the analyzer costs one LLM call per tool invocation ( $\approx 800$  ms median, cached on replay); the schema-only path remains available as the low-latency default.

g) *Framework comparison and latency.*: All three framework adapters (AutoGen, MCP, LangChain) produce identical privacy results; adapter overhead is  $< 0.1$  ms. Median latency is 1.77 ms; P99 is 10.8 s without timeout, inflated by deeply nested JSON ( $> 5,000$  chars) rather than catastrophic backtracking. A 2 s classification timeout caps worst-case latency. Latency scales linearly with argument size. The sub-2 ms schema-only path stays negligible inside multi-step agent loops: a 10-step ReAct trajectory adds  $\approx 18$  ms of middleware latency in total, orders of magnitude below the per-step LLM inference it gates; the heavier content-aware layer ( $\approx 800$  ms/call) is applied selectively to free-text fields rather than at every step.

h) *Metric validation.*: Our primary validation is ranking stability: across 110 perturbations of the  $S$  and  $E$  weight mappings, TOOLMINIMIZE is the lowest-PCS utility-preserving method in 100% of parameterizations (Kendall’s  $\tau = 1.0$ ). The comparison result in Section V-C therefore does not depend on the specific weight choice. As a sanity check, three LLM judges (GPT-4o, Claude Sonnet, Llama-3.3-70B) rated all 73 Stage C calls on a harm/re-identification/sensitivity rubric (1–5); judges agree with each other (mean pairwise  $\rho = 0.69$ ) but rate a privacy-critical corpus at ceiling (range 2.2–5.0, median 4.3). Under schema-level  $N$ , 18/73 calls score PC = 0 (all send\_email and send\_message: every detected item lies in a minimum\_necessary field), which

yields  $\rho = -0.26$  ( $p = 0.03$ ) – a zero-variance artifact, not an inverse relationship. Per-item  $N$  judged by GPT-4o removes the collapse and recovers  $\rho = +0.04$  ( $p = 0.74$ , 95% CI  $[-0.20, +0.28]$ ); per-call correlation cannot climb higher against ceiling-saturated judges, but the point estimate is non-negative as required. Discussion in Section VI-C.

i) *End-to-end execution and linkability.*: On 20 scenarios against mock endpoints, 16/20 (80%) produce functionally equivalent results; all four failures are maps scenarios where required addresses were removed for lack of minimum\_necessary annotations. On 13 multi-call scenarios, TOOLMINIMIZE reduces cross-call PSD linkability by 70.6% (17→5 shared items); session-level budgets would reduce this further (Section VI-D0b).

j) *Error analysis and recall caveat.*: Three failure modes account for residual PCS: missed DI (regex recall 0.435; NER raises to 0.739), necessary PSD retained (3/90 scenarios), and over-generalization (2 scenarios). Headline reductions are over detected PSD; missed items propagate unchanged, so true leak suppression is bounded above by classifier recall.

## E. Summary

Over-sharing is pervasive ( $> 99\%$ ); schema-only TOOLMINIMIZE achieves 81.2–92.0% on live LLM outputs (mean 88.9%), 89.1% on 50 external scenarios, 79.0% on 25 unannotated MCP schemas, and 73.6% on author schemas. The optional content-aware extension raises live-LLM reduction to 85.1–95.6% (mean 91.6%) and author-schema reduction to 90.9% by stripping task-irrelevant PSD from otherwise-necessary free-text fields. Each pipeline stage contributes meaningfully; latency scales linearly; all three framework adapters produce identical results.

## VI. DISCUSSION

### A. Implications and Recommendations

Over-sharing is structural: 81–88% of real LLM tool calls include unnecessary PSD. Prompting reduces this by at most 56% (GPT-4o) and as little as 6% (Llama). Schema-aware middleware achieves 73.6% with full task validity. The key distinction is between schema-required fields and functionally necessary content: a weather API requires a location field but needs only city-level precision.

a) *Developers.*: Deploy argument-level privacy middleware as a standard pipeline component; annotate tool schemas with minimum\_necessary fields (5–15 min per tool); default to THIRD\_PARTY\_NO\_DPA for over-minimization. Tools requiring high-precision data (maps) fail without annotations, motivating community-maintained schema registries.

b) *Scaling beyond curated handlers.*: The tool-type handlers (weather, calendar, maps, ...) are an *optimization*, not a requirement. A single generic handler—generalize critical-sensitivity free text, drop optional PSD fields—already covers 62.2% of scenarios, and the content-aware layer is fully tool-agnostic, judging per-item necessity from the field value and task alone. On 25 real-world MCP schemas with no custom handlers, TOOLMINIMIZE still reduces cost by 79.0%

(Section V-D). Handlers improve precision on common tools but are not needed to onboard the long tail of APIs.

c) *Protocol designers.*: MCP and other protocols currently lack privacy metadata. We propose *per-argument privacy labels*, *tool-level trust declarations* in server manifests, and *privacy negotiation* for data retention. Agent-to-agent protocols should include a `required_context` field; our Stage E evaluation shows TOOLMINIMIZE reduces cross-agent PCS from 12.60 to 0.00.

### B. Argument Validity vs. Task Utility

TCR measures *argument-level* validity—rewritten arguments stay schema-valid and keep every minimum-necessary field—not end-to-end task success, and the two can diverge: generalizing a calendar event’s location or stripping a maps destination can yield a valid call that no longer serves the user’s intent. We therefore also report end-to-end execution against mock endpoints (Section V-D): 16 of 20 scenarios (80%) return functionally equivalent results, and all four failures are maps scenarios whose required addresses were removed absent `minimum_necessary` annotations—exactly the case where the over-shared field is also the task payload. Three mechanisms narrow this gap: the content-aware layer strips task-irrelevant PSD from *within* an otherwise-necessary free-text field instead of dropping the field; the optional `task_context` parameter retains user-consented PSD; and *fail-prompt* mode surfaces borderline rewrites for confirmation. Location-precise tools remain the hard case: when the destination is itself the sensitive item, routing and minimization conflict, and annotation-guided generalization (neighborhood- rather than facility-level) is the most that is safe without a user in the loop. Distinguishing the field that is structurally required from the content within it that is task-essential is the central design tension of argument-level minimization.

### C. Metric Validation

PC is designed to rank methods by how much exposure they remove, not to predict per-call judge severity. The primary validation is therefore ordinal: *across 110 perturbations of the S and E weight mappings, the method ranking is identical (Kendall’s  $\tau=1.0$ ); TOOLMINIMIZE is the best utility-preserving method in 100% of parameterizations*. This is the property the comparison results in Section V-C actually depend on, and it holds regardless of the specific weight choice.

Per-call correlation with LLM judge severity is a secondary sanity check. Three LLM judges (GPT-4o, Claude Sonnet, Llama-3.3-70B) agree with each other ( $\rho=0.69$ ) but rate a privacy-critical corpus near ceiling (median 4.3/5), so even a perfect metric could not produce strong rank correlation against compressed judge scores. Under schema-level  $N$  the aggregate  $\rho=-0.26$  ( $p=0.03$ ), driven entirely by 18/73 messaging calls receiving PC=0 when all PSD lives in `minimum_necessary` fields; per-item  $N$  judged by an LLM removes the zero-variance collapse and recovers

$\rho=+0.04$  (95% CI  $[-0.20, +0.28]$ ). The point estimate is non-negative, which is what the metric needs given saturation. The ranking-monotonicity claim is what carries the comparison.

### D. Limitations, Error Analysis, and Future Work

a) *Limitations.*: *Simulated vs. live*: live validation on 307 tool calls shows no significant difference (Mann-Whitney  $p=0.085$ ; TOST  $p<0.001$  at  $\Delta=1.0$ , not at  $\Delta=0.25$ ). *TCR scope*: schema-validity, not end-to-end (80%; 4/20 maps failures from missing annotations; external TCR drops to 60%). *Ecological validity*: the 99.7% baseline PER reflects a privacy-critical corpus; general use is lower. *User agency*: TOOLMINIMIZE decides unilaterally; classifier targets US formats; generalization may signal concealment. *Defense*: 1/10 encodings (field splitting) still evades; 10 categories cover GDPR Art. 9 but not all special categories. *False positives*: on benign inputs detection fires on 20% and rewriting on 10% (Section V-D)—an over-cautious bias, mitigated by *fail-prompt* confirmation, the per-item content-aware check that rejects spurious rewrites, and confidence thresholds. Error analysis: missed DI ( $\sim 40\%$  of residual), necessary PSD retained (3/90), over-generalization (2).

b) *Future work.*: Adaptive NER backend for latency; session-level budgets [16]; CaMeL integration for joint confidentiality–integrity [8]; user-controlled minimization with preview UI; multimodal PSD detection.

## VII. RELATED WORK

a) *Privacy in LLM systems.*: Prior work addresses training-time memorization [17] and conversational disclosure [23], [22], [24]. Those adversaries extract data *from* the model; tool call over-sharing is the model actively *sending* data to third parties.

b) *Agent security and privacy.*: AudAgent [3] auto-formalizes natural-language privacy policies via cross-LLM voting and blocks policy-violating actions, but its intervention is binary (allow/block) at the action level rather than field-level rewriting. PrivacyChecker [4] extracts the contextual-integrity (CI) five-tuple [25] per information flow and gates entire tool invocations; it reasons about attributes but does not mutate argument values, so it cannot distinguish a weather query with “New York, NY” from one with “Memorial Sloan Kettering.” Progent [5] also uses JSON Schema to express fine-grained policies over tool call arguments, but enforces via allow/block with fallback actions rather than argument-level rewriting. Das et al. [26] audit contextual privacy with detection focus. AgentPrint [27], MEXTRA [28], and Patil et al. [16] address traffic fingerprinting, memory extraction, and compositional risks, respectively. Chen et al. [29] show users systematically underestimate privacy leakage; the invisibility of tool call arguments exacerbates this. Concurrent benchmarks [9], [10], [11] corroborate over-sharing in agent pipelines.

c) *Information flow control.*: Agent IFC systems track confidentiality and integrity labels at flow granularity. Fides [6] tracks both axes with capability-based allow/deny decisions; CaMeL [8] uses capabilities to block private-data exfiltration



over unauthorized flows; RTBAS [7] targets both prompt injection and privacy leakage. All three enforce at flow granularity rather than performing graduated rewriting (removal, generalization, truncation) of individual argument fields, and their binary tainted/clean labels cannot capture the sensitivity spectrum (an email address and a cancer diagnosis carry very different exposure risks). TOOLMINIMIZE and these systems are complementary.

*d) Data minimization and PII tools.*: GDPR Art. 5(1)(c) [21] codifies minimization; Presidio and AWS Comprehend implement it via pattern+NER. These tools have three limitations for agent tool calls: (i) they miss *implicit* PSD (“Memorial Sloan Kettering” is not PII but implies cancer); (ii) they are *schema-unaware*; (iii) they apply uniform redaction regardless of tool trust. GAMA [30] and DP-RAG [31], [32] handle document-level anonymization; PAPILLON [33] and Bagdasarian et al. [34] minimize at the prompt boundary; PrivWeb [35] protects DOM text for web agents. Differential privacy (DP) is a poor fit for tool arguments: format constraints (email, ISO date) preclude noise. TOOLMINIMIZE targets a complementary boundary (tool call argument egress) with schema-based necessity and graduated rewriting (Table VI).

No existing system performs schema-aware, argument-level minimization by rewriting tool call fields while preserving task validity; TOOLMINIMIZE fills this gap. Integration with CaMeL or Progent for joint confidentiality–integrity is feasible (Section VI-D0b).

## VIII. CONCLUSION

Safety training cannot eliminate tool call over-sharing: the helpfulness that makes an agent useful is the same mechanism that leaks context into its tool arguments. TOOLMINIMIZE shows that schema-aware argument rewriting reduces privacy cost by **81.2–92.0% on 307 live tool calls** from GPT-4o, Claude Sonnet, and Llama-3.3-70B, with 100% argument-level task validity (TOST  $p < 0.001$ ). The schema-only path runs at 1.77 ms median, no LLM call required. An optional content-necessity layer that asks an LLM, per detected item, whether the value is task-essential raises live-LLM reduction to 85.1–95.6% and author-schema reduction from 71.1% to 90.9% by catching PSD that lives inside otherwise-necessary free-text fields. Residual exposure is bounded by classifier recall (worst case 4.09 PCS/scenario, observed 0.59). Cross-session aggregation, adversarial field-splitting, and multimodal PSD remain open problems for agent-privacy middleware.

## REFERENCES

- [1] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: <https://arxiv.org/abs/2302.04761>
- [2] C. Qin, A. Zhang, A. D. Chen, W. Ye, Z. Zhang et al., “Tool learning with large language models: A survey,” *arXiv preprint arXiv:2405.17935*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.17935>
- [3] Y. Zheng, Q. Liu, Y. Wang, X. Li, and X. Chen, “AudAgent: Automated auditing of privacy policy compliance in AI agents,” *arXiv preprint arXiv:2511.07441*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.07441>
- [4] H. Chen, Y. Li, R. Yan, and N. Mireshghallah, “Privacy in action: Towards realistic privacy mitigation and evaluation for LLM-powered agents,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, arXiv:2509.17488. Introduces PrivacyChecker and PrivacyLens-Live. [Online]. Available: <https://arxiv.org/abs/2509.17488>
- [5] T. Shi et al., “Progent: Programmable privilege control for LLM agents,” *arXiv preprint arXiv:2504.11703*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.11703>
- [6] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Securing AI agents with information-flow control,” *arXiv preprint arXiv:2505.23643*, 2025, introduces Fides planner. Microsoft Research. [Online]. Available: <https://arxiv.org/abs/2505.23643>
- [7] P. Y. Zhong, S. Chen, R. Wang, M. McCall, B. L. Titzer, H. Miller, and P. B. Gibbons, “RTBAS: Defending LLM agents against prompt injection and privacy leakage,” *arXiv preprint arXiv:2502.08966*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.08966>
- [8] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” *arXiv preprint arXiv:2503.18813*, 2025, introduces CaMeL. Google/ETH Zurich. [Online]. Available: <https://arxiv.org/abs/2503.18813>
- [9] H. Trivedi, L. Bauer, and R. Jia, “AgentDAM: Privacy leakage evaluation of web agents via data minimization,” in *Proceedings of the NeurIPS 2025 Datasets and Benchmarks Track*, 2025, arXiv:2503.09780.
- [10] Z. Han et al., “Agent tools orchestration leaks more than you think,” *arXiv preprint arXiv:2512.16310*, 2025.
- [11] J. Kang et al., “AgentLeak: A benchmark for privacy leakage in multi-agent systems,” *arXiv preprint arXiv:2602.11510*, 2026.
- [12] F. He et al., “The emerged security and privacy of LLM agent: A survey with case studies,” *ACM Computing Surveys*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.19354>
- [13] B. Radosevich and J. Halloran, “MCP safety audit: LLMs with the model context protocol allow major security exploits,” *arXiv preprint arXiv:2504.03767*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.03767>
- [14] X. Hou et al., “Model context protocol (MCP): Landscape, security threats, and future research directions,” *arXiv preprint arXiv:2503.23278*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.23278>
- [15] H. Errico, D. Kim, and J. Park, “Securing the model context protocol (MCP): Risks, controls, and governance,” *arXiv preprint arXiv:2511.20920*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.20920>
- [16] D. J. Patil, E. Stengel-Eskin, and M. Bansal, “The sum leaks more than its parts: Compositional privacy risks and mitigations in multi-agent collaboration,” *arXiv preprint arXiv:2509.14284*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.14284>
- [17] A. Panda, X. Tang, M. Nasr, C. A. Choquette-Choo, and P. Mittal, “Privacy auditing of large language models,” in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2503.06808. [Online]. Available: <https://arxiv.org/abs/2503.06808>
- [18] C. Schroeder de Witt, J. Foerster et al., “Open challenges in multi-agent security: Towards secure systems of interacting AI agents,” *arXiv preprint arXiv:2505.02077*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.02077>
- [19] Z. Li et al., “IP leakage attacks targeting LLM-based multi-agent systems,” *arXiv preprint arXiv:2505.12442*, 2025, introduces MASLEAK framework. [Online]. Available: <https://arxiv.org/abs/2505.12442>
- [20] L. Sweeney, “k-anonymity: A model for protecting privacy,” *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557–570, 2002.
- [21] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 of the European Parliament and of the Council (general data protection regulation),” *Official Journal of the European Union*, 2016, l 119, 4 May 2016.
- [22] Y. Li, R. Yan, N. Mireshghallah, and Y. Tsvetkov, “PrivacyLens: Evaluating privacy norm awareness of language models in action,” in *Advances in Neural Information Processing Systems (NeurIPS)*

TABLE VI

COMPARISON WITH RELATED SYSTEMS. GRAN. = GRANULARITY; IMPL. = IMPLICIT PSD; SCH. = SCHEMA-AWARE; REWR. = REWRITING; QUANT. = METRIC; UTIL. = UTILITY; INTEG. = INTEGRITY. ✓ = SUPPORTED, ○ = PARTIAL, – = NOT.

| System              | Gran.       | Impl.    | Sch.     | Rewr.    | Quant.   | Util.    | Integ.   | Focus          |
|---------------------|-------------|----------|----------|----------|----------|----------|----------|----------------|
| AudAgent            | Flow        | –        | –        | –        | –        | n/a      | –        | Det./block     |
| PrivacyChecker      | Flow        | ○        | –        | –        | –        | Low      | –        | CI gate        |
| Progent             | Arg.        | –        | ✓        | –        | –        | ○        | –        | Priv. ctrl     |
| Presidio            | Tok.        | –        | –        | ○        | –        | ○        | –        | PII redact.    |
| Fides               | Flow        | –        | –        | –        | –        | n/a      | ✓        | IFC            |
| RTBAS               | Flow        | –        | –        | –        | –        | n/a      | ✓        | IFC            |
| CaMeL               | Flow        | –        | –        | –        | –        | ✓        | ✓        | IFC            |
| <b>ToolMinimize</b> | <b>Arg.</b> | <b>✓</b> | <b>✓</b> | <b>✓</b> | <b>✓</b> | <b>✓</b> | <b>–</b> | <b>Confid.</b> |

*Datasets and Benchmarks Track*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.00138>

- [23] N. Mireshghallah, H. Kim, X. Zhou, Y. Tsvetkov, M. Sap, R. Shokri, and Y. Choi, “Can LLMs keep a secret? testing privacy implications of language models via contextual integrity theory,” in *International Conference on Learning Representations (ICLR)*, 2024, iCLR 2024 Spotlight. Introduces ConfAide benchmark. [Online]. Available: <https://arxiv.org/abs/2310.17884>
- [24] H. Li *et al.*, “PrivaCI-Bench: Evaluating privacy with contextual integrity and legal compliance,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.17041>
- [25] H. Nissenbaum, *Privacy in Context: Technology, Policy, and the Integrity of Social Life*. Stanford University Press, 2010.
- [26] S. Das, J. Sandler, and F. Fioretto, “Beyond jailbreaking: Auditing contextual privacy in LLM agents,” *arXiv preprint arXiv:2506.10171*, 2025, neurIPS MTI-LLM Workshop 2025. [Online]. Available: <https://arxiv.org/abs/2506.10171>
- [27] Y. Zhang, X. Deng, Z. Gu, Y. Chen, K. Xu, Q. Li, and J. Wu, “Exposing LLM user privacy via traffic fingerprint analysis: A study of privacy risks in LLM agent interactions,” *arXiv preprint arXiv:2510.07176*, 2025, introduces AgentPrint. [Online]. Available: <https://arxiv.org/abs/2510.07176>
- [28] B. Wang, W. He, S. Zeng, Z. Xiang, Y. Xing, J. Tang, and P. He, “Unveiling privacy risks in LLM agent memory,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, arXiv:2502.13172. Introduces MEXTRA attack. [Online]. Available: <https://arxiv.org/abs/2502.13172>
- [29] Z. Chen, T. Zhang, and N. Gong, “Privacy leakage overshadowed by views of AI: A study on human oversight of privacy in language model agent,” *arXiv preprint arXiv:2411.01344*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.01344>
- [30] H. Yang *et al.*, “GAMA: A general anonymizing multi-agent system for privacy preservation enhanced by domain rules and disproof mechanism,” *arXiv preprint arXiv:2509.10018*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.10018>
- [31] T. Koga, R. Wu, Z. Zhang, and K. Chaudhuri, “Privacy-preserving retrieval-augmented generation with differential privacy,” *arXiv preprint arXiv:2412.04697*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.04697>
- [32] N. Grislain, “RAG with differential privacy,” *arXiv preprint arXiv:2412.19291*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.19291>
- [33] L. Siyan *et al.*, “PAPILLON: Privacy preservation from internet-based and local language model ensembles,” in *Proceedings of the 2025 Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025. [Online]. Available: <https://arxiv.org/abs/2410.17127>
- [34] E. Bagdasarian *et al.*, “Operationalizing data minimization for LLM prompting,” *arXiv preprint arXiv:2510.03662*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.03662>
- [35] Z. Yang *et al.*, “PrivWeb: Content-aware privacy protection for web agents,” *arXiv preprint arXiv:2509.11939*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.11939>